LLM-Assisted Program Analysis for Vulnerability Chain Detection

UC SANTA BARBARA Early Research Scholars Program

Mahima Chaudhary, Vishva Arasan, Wendy Contreras Martinez, Sahasra Boddula, Daniil Novak

Advisors: Professor Tevfik Bultan, Md Shafiuzzaman

Background: Analysis, Execution, Chaining

- **Static Analysis**: analyzes source code against queries to capture vulnerability patterns.
- **Symbolic Execution:** Use symbolic inputs to explore multiple execution paths to identify complex vulnerabilities
- **LLM Harness Generation**: Automatically generate symbolic execution harnesses from source code.
- **Bug Triage**: Prioritizes and filters static analysis results before passing them to symbolic execution, targeting only the specs most likely to yield real vulnerabilities.
- **Chaining**: Sequences multiple bugs together so that the confirmed crash state of one vulnerability becomes the entry point for another.

Problem: Complexity in Deep Exploration

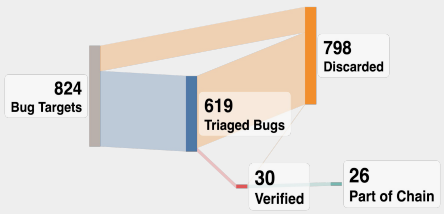
Problem:

- Static analysis: Too many false positives → wastes time on non-vulnerabilities
- Symbolic execution: Requires expert knowledge → hard for non-specialists to use
- LLMs: Can hallucinate → need a feedback/verification loop
- Individual bugs: Often deprioritized alone → chaining reveals true impact

Goal: Design an automated vulnerability detection pipeline accessible to undergraduates and non-specialists. Incorporate custom queries to improve detection of complex vulnerability specifications, bug chaining to enable deeper path exploration, and bug triage to minimize computation time and cost.

Baseline Testing and Triage on TCPDump

Figure 1: Example Run on TCP Dump



Through our triaged pipeline we were able to:

- Reduce API Consumption by **90%.**
- Speed up execution time by **10 times.**
- Identify **20-40%** of the Chainable bugs.

Automated Vulnerability Discovery Pipeline

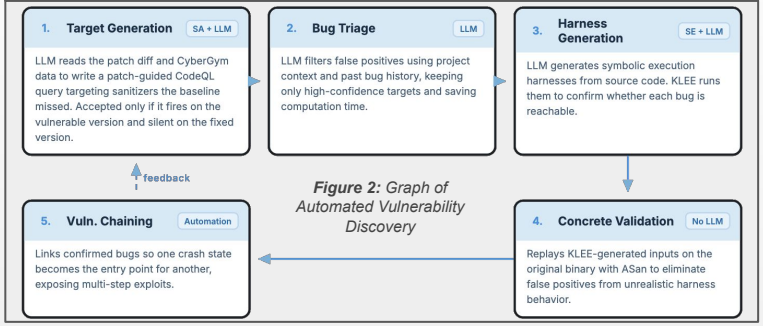


Figure 2: Graph of Automated Vulnerability Discovery

LLM-Automated Query Generation

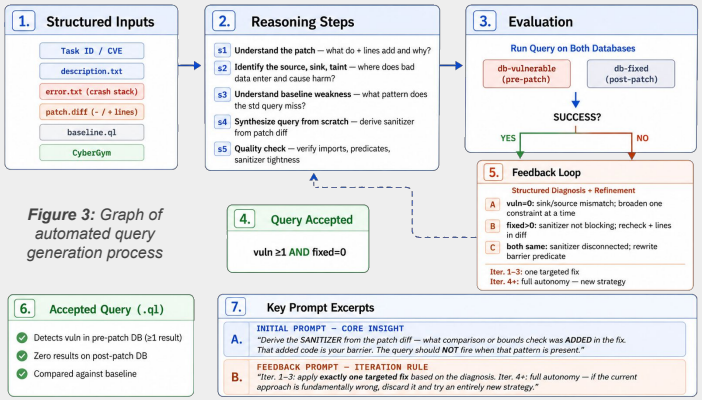
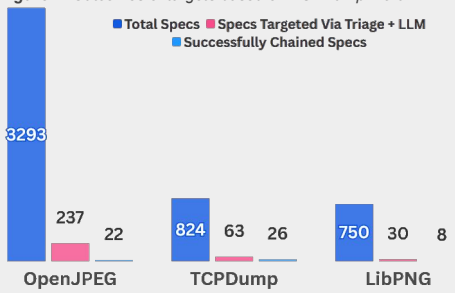


Figure 3: Graph of automated query generation process

Experimental Results Across Projects

Figure 4: Outcomes of targets based on TCPDump 4.9.0



Over **3 projects** with **570k+ LOC** analyzed.

Our pipeline was able to find at least **2 chains** in each project.

We noticed that in larger codebases our pipeline was able to identify at least **~20%** more chainable vulnerabilities, while not necessarily having more standalone bugs.

Validated Bug Chains and Query Performance

Figure 5: TCPDump Call-Graph Reachability Results

CALL-GRAPH FINDING	FINAL RESULT	WHAT IT MEANS
Confirmed bugs analyzed	16 (confirmed bugs)	The call graph was evaluated across 16 confirmed bug entries.
Entry-to-entry reachable pairs	86 (entry paths)	86 ordered bug pairs had a path where one bug's entry could transitively call another bug's entry.
Entry-to-crash reachable pairs	82 (crash paths)	82 ordered bug pairs reached the other bug's actual crash function, the stronger chain signal.
Overall crash-chain coverage	82 / 240 (possible directed pairs)	Out of all possible directed bug pairs, 82 showed crash-function reachability.

Figure 6: Example of Concrete Validation of Theoretical Chains from Call-Graph

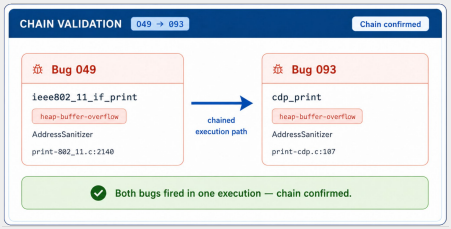


Figure 7: Synthesized queries detected 3/3 CVEs; baseline detected 0/3. Zero false positives on patched code.

Each cell: **results on vuln DB** / label = result on fixed DB · CodeQL 2.25.3 · CyberGym Benchmark — Model: Claude Opus 4.7

PROJECT / CWE	BASELINE detections on vuln DB pre-written query	LLM-SYNTHESIZED detections on vuln DB patch-guided query	ITERS TO PASS
pcre2 CWE-125 · Out-of-Bounds Read	0 not detected	1 0 FP on fixed DB	2
miniz CWE-457 · Uninit. Variable	0 not detected	1 0 FP on fixed DB	5
libxml2 CWE-476 · NULL Ptr Dereference	0 not detected	2 0 FP on fixed DB	1
Overall	0 / 3 detected by baseline	3 / 3 detected by synthesized queries	

Key Takeaways & Future Work

Custom Queries: Generates custom CodeQL queries for complex vulnerability specifications, improving detection beyond baseline pre-written queries.

Bug triage: Reduces hallucinations by filtering low-confidence results and prioritizing high-confidence vulnerabilities for review.

Chaining: Uses symbolic execution to explore deeper vulnerable execution states and reveal how one confirmed bug can lead to another.

Future work: Improve query precision, further reduce false positives, and expand support to larger, more complex repositories.

Acknowledgements: We would like to thank our advisors Professor Bultan and Md Shafiuzzaman, as well as our ERSP advisors Professor Ziad Matni and Parker Carlson.

References:

[1] Shafiuzzaman et al. 2026. Guiding Symbolic Execution with Static Analysis and LLMs for Vulnerability Discovery.

[2] Shafiuzzaman et al. 2024. STASE: Static Analysis Guided Symbolic Execution for UEFI Vulnerability Signature Generation. ASE '24. https://doi.org/10.1145/3691620.3695543